

Computing:

We are at the dawn of an era that transcends CPU and GPU computing

Open computer vision (OpenCV) is an image-processing software that is widely used and heavily documented. With the latest version having participation from big players like Intel and AMD, let us find out about the open computer language (OpenCL) acceleration layer added to it. Dr Harris Gasparakis, OpenCV manager, Computer Vision, AMD, speaks with Priya Ravindran from EFY about graphical processing and AMD's contribution to OpenCV



**DR HARRIS
GASPARAKIS**
OPENCV MANAGER,
COMPUTER VISION, AMD

Q. How has computing evolved?

A. The modern trend embodied by graphical processing units (GPUs) is to employ scalar architectures, support multiple concurrent threads and let the compiler and runtime do their magic. In fact, we are at the dawn of an era that transcends central processing unit (CPU) and GPU computing by combining both. I am talking about heterogeneous computing, where CPUs and GPUs are not only integrated on the same die but these efficiently synchronise access and operate on the same data from main memory, without redundant data copies. OpenCL 2.0 provides the application peripheral interface that enables this.

Q. What are the main points to keep in mind while creating an OpenCL implementation?

A. There are two main elements in creating an OpenCL implementation or port of any library. First, you need to make sure that data makes it to the processing cores. For example, in a discrete GPU, you need to explicitly copy the data to onboard memory from the main memory. This approach also works on an integrated device but is not efficient.

The other element in porting a library to OpenCL is porting the processing logic to OpenCL kernel syntax.

Q. What are the main differences between OpenCV-without-OpenCL and OpenCV-with-OpenCL? What new capabilities do these offer?

A. It is not about new functionalities. Rather, it is about accelerating the most common existing functionality to take advantage of GPUs. For copying data to the onboard memory on an integrated GPU, you would want to use zero copy, that is, you would want the GPU to use the same data as the CPU. This was possible using OpenCL 1.2 but is significantly easier with OpenCL 2.0. In OpenCV, our goal was to create an implementation that would work on all OpenCL-capable devices, such as integrated or discreet GPUs.

Q. Why did you go with the transparent acceleration layer?

A. The goal of the transparent acceleration layer (T-API)

was to enable people to write their code only once. If an OpenCL-capable device is available, it will be used. Otherwise, the fallback is CPU execution, which can also include accelerators like integrated performance primitives or intrinsics like advanced vector extensions/streaming SIMD extensions (AVX/SSE).

Detection of OpenCL devices happens at runtime, dynamically. Integration inside OpenCV was a significant effort. We sponsored the maintainers of OpenCV and they became very excited with the vision and carried it forward.

Q. How does this interface work?

A. My main idea was to introduce a new data structure, universal or unified matrix (Umat) to replace Mat data structure, which was historically the basic image data structure in OpenCV. The goal was to hide data locality under UMat hood in a way that is appropriate for each class of device. For example, in the case of discreet GPUs, UMat would have the responsibility for synchronising data between the CPU and GPU by making a copy.

Q. To what effect can embedded engineers use OpenCV?

A. Traditionally, OpenCV has been great for prototyping, given the wealth of algorithms that have been implemented. OpenCV 3.0 brought about a general re-architecture, further strengthening the library in terms of having it organised in a more modular fashion.

Embedded vendors have now realised that there are credible, high-performance and relatively low-cost embedded solutions using GPUs. In the cost, one has to factor in the total cost of ownership, as developing in generic languages like OpenCL is arguably significantly cheaper than developing for a specialised processor.

Q. What is the next step forward for OpenCV?

A. The next step in OpenCV will be to take full advantage of the advanced features of OpenCL 2.0. I am particularly excited about the prospect of extensively using fine-grain support vector machine (SVM), as this would enable efficient hybrid imaging pipelines, where one would be able to mix and match CPU and GPU execution, with no performance penalty. ●